🌡️ Waterproof Temperature Sensor (DS18B20)

Hardware Overview

🌡️ What is it?

The **DS18B20** is a highly accurate temperature sensor sealed inside a waterproof probe. It is suitable for measuring the temperature of liquids, soil, or harsh environments from `-55°C` to `+125°C`.

⚡ Electrical Explanation

This sensor uses a communication protocol called *1-Wire*. Unlike I²C (which requires two data lines) 1-Wire allows the controller to communicate with the sensor using only *a single digital pin*.

- Wires: The sensor has `3` pins: **Black (`GND/0V`)**, **Red (`Power/5V`)**, and **Yellow (data wire, e.g. `D8`)** + `4.7kΩ` resistor connected to `5V`.

⚠️ **The Pull-up Resistor** Because the single data wire must transmit data in both directions, it requires "help" to return to a "High" state. You MUST place a `4.7kΩ` resistor connecting the data wire and the `5V` wire. If you forget this resistor, the sensor will never communicate!

Key Hardware Facts:

- **Connection:** Wired to a Digital Pin (e.g. `D12`).
- **Voltage:** Operates at **`5V`**.
- **Signal Flow:** Ambient Heat ➔ Waterproof Casing ➔ Silicon Temperature Sensor ➔ ADC ➔ 1-Wire Data ➔ Processor.

Software & Programming

📚 `kcomp_DS18B20.h`

This library hides the timings of the *1-Wire* bus, allowing you to request and read temperature data.

Usage: `#include <kcomp_DS18B20.h>`

📦 The Data Structure

Unlike other sensors that give you a single number (like `int`), the DS18B20 gives you a `struct` (a package of variables). When you call `getDS18B20Data()`, you get a package containing all of this:

Field	Type	Description
`is_valid`	**`bool`**	`true` if the temperature was successfully read and verified.
`temperature_c`	**`float_t`**	The temperature of the probe in `°C`.

Function Reference

Function Name	Description & Parameters
`bool initDS18B20(uint8_t pin)`	**Setup:** Starts the *1-Wire* bus on a specific digital pin. ▪ Returns `true` if connected, `false` if missing.
`void requestDS18B20()`	**Trigger:** Wakes the sensor up and asks for the temperature.
`ds18b20_data_t readDS18B20()`	**Read Data:** Retrieves the data after a `750ms` delay. If `is_valid` is `false`, it is still calculating.

⚠️ Pitfalls & Troubleshooting

- **The Missing Resistor**
  If you forget about the `4.7kΩ` resistor, the sensor will not work.
  The Fix: Simply do not forget about the `4.7kΩ` resistor!
- **The "Highlander" Rule (There Can Be Only One!)**
  To keep this library lightweight, it is designed as a "Singleton". It only supports one sensor.
  The Problem: To make our library flat and easy to read, it uses a command called `SKIP_ROM (0xCC)`. This tells all sensors: "*I don't care what your ID is, everyone measure the temperature!*" Thus, if you use multiple sensors, they will corrupt the data.
  The Fix: Stick to one DS18B20 thermometer per project!
- **Waterproof vs. Submersible**
  The metal tip of the sensor is waterproof. However, the black heat-shrink is only *water-resistant*. If you submerge the entire wire and the heat-shrink in a pot of boiling water, the adhesive inside the tubing will melt, water will leak in, and the sensor will be destroyed.
  The Fix: Only submerge the silver metal part in boiling hot water!

📝 Sample Program

It takes the sensor about `750ms` to calculate the temperature. Using `delay()` would block your program. We can do better: this code demonstrates a **non-blocking** loop!

```
#include <kcomp_DS18B20.h>

// We are connecting the Yellow Data wire to Digital Pin 12
#define TEMP_PIN 12

void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println("K-Comp Thermometer Online");

  if (!initDS18B20(TEMP_PIN)) {
    Serial.println("DS18B20 not found! 4.7k resistor betw. 5V and DQ?");
    while(1);
  }

  requestDS18B20(); // Ask the sensor to start measuring
}

void loop() {
  // Check if the sensor has finished its 750ms calculation
  ds18b20_data_t temp_sensor = readDS18B20();

  if (temp_sensor.is_valid) {
    Serial.print("Temperature: ");
    Serial.print(temp_sensor.temperature_c); Serial.println(" °C");
    // Immediately ask it to start the NEXT measurement
    requestDS18B20();
  }
  // Your K-Comp is free to run other code without blocking!
}
```

💡 **Try it out:** You can find the full sample program **`SimpleTemperatureSensorDS18B20`** in the K-Comp examples folder!